The Volume of the Knot Potential: Decay and Direction on the Vacuum Disk

Jeromie Beasley

7 November 2026 (v1)

AI-assisted: computations, drafting and literature checks were carried out with Claude (Anthropic) and helper agents, from the author's ideas and earlier papers; all numbers come from scripts listed in Appendix C and the attached archive.

Abstract

VOTKP began as the *volume of the knot potential*, an attempt to read the volume of the space around a knot from its polynomials. It then became the *vector of traits kept under peeling*. This paper joins the two meanings through the two books of the intrinsic quantum geometric tensor $Q = g - \frac{i}{2}\Omega$: a climb (metric, size) and a turn (curvature, phase). The reading is set on a Smith-type disk whose centre is the vacuum and whose rim is ground.

- **The vacuum disk.** Every ideal tetrahedron of a hyperbolic knot complement is placed on a disk centred on the regular tetrahedron. On that disk:
  - the three ways of reading a tetrahedron are exact 120° turns, which is the tennis-racket switch: the axis changes, the volume does not;
  - flat tetrahedra lie on the rim, and mirrors lie outside it.
- **Decay.** A tetrahedron's volume is a bowl around the vacuum, with the exact series $D = V_0 - \frac{3\sqrt{3}}{4}|G|^2 + \frac{\sqrt{3}}{8}(G^3 + \bar{G}^3) + \frac{3\sqrt{3}}{16}|G|^4 + O(|G|^5)$. Summed over the pieces, with nothing fitted, it gives the volume of 12,954 knots to a median error of 0.076%.
- **Direction.** The Chern–Simons twist lives in integer windings. With the pieces read in a branching order it is reproduced exactly (454 of 454 runs).
- **Our polynomial.** The knot potential read straight off a drawing gives one (decay, direction) root per geometry.
  - On 79 of 79 hyperbolic knots, its top root is the knot's complex volume.
  - Its spectrum is the same from different drawings (13 of 13 knots tested).
  - The true geometry is always the root nearest the vacuum (31 of 31).
  - Torus knots give pure-direction roots: zero decay, a rational fraction of a turn. The knots that assemble themselves in chemistry are exactly of this kind.
- **Supporting lines.** Light strings (Jones, HOMFLY along its sl(N) lines), the colour of the elements on the same disk, and the electron's group $\mathrm{SL}(2,\mathbb{C})$ complete the chain.
- **Status.** Reproductions of known mathematics, new readings and new results are marked throughout and listed in Section 12.

1. Where this comes from

Three papers feed this one.

- **IQGT** (*Intrinsic Quantum Geometric Tensor*, v7.3.1). For a projector path, the tensor $Q = g - \frac{i}{2}\Omega$ splits into two books.
  - The metric $g = \frac{1}{2}\mathrm{Tr}(VW)$ measures how far a state moves: the *climb*.
  - The curvature $\Omega = +i\,\mathrm{Tr}(\Pi[V, W]) = -2\,\mathrm{Im}\,Q$ measures how it turns: the *turn*.
  - A loop that comes back still remembers, through its phase (IQGT Ex. 6.5).

  - The Smith coordinate $\Gamma = (z-1)/(z+1) = \tanh(\frac{1}{2}\ln z)$ puts any impedance-like quantity on a disk: the centre $\Gamma = 0$ is the matched point (vacuum) and the rim $|\Gamma| = 1$ is total reflection (ground).
- **Dirac Time** (*Dirac Time of the Gigantefermion*, v3.1). Torus knots, their arcs and signatures, and the chiral pairing $\Gamma P_+ \Gamma = P_-$. The torus knots return here as the pure-direction roots.
- **VOTKP** (*Elemental Peeling*, v2.9). Peeling kept a vector of traits, Levine–Tristram walls and colour; v2.9 Table 6 has 575 double roots. Its first meaning, the volume of the knot potential, mixing HOMFLY and coloured Jones to map volume, is the question this paper answers.

The tennis racket. A body spun about its middle axis flips over and over (the Dzhanibekov effect). The angular momentum keeps its size and its direction in space. What builds up is a turn, a geometric phase set by the area the switching sweeps (Montgomery). In an earlier part of this research the climb rate of that switch matched the Connection Energy formula (0.577350), and the memory was a lune $4\arctan k$ that reaches 45° when the middle level sits midway. The rule *the axis switches, the direction stays* is the organising rule of everything below.

2. The vacuum disk for knot pieces

A hyperbolic knot complement is glued from ideal tetrahedra. Each has a shape z with $\operatorname{Im} z > 0$, a cross-ratio of its four ideal vertices. Its volume is the Bloch–Wigner dilogarithm

$$D(z) = \operatorname{Im} \operatorname{Li}_2(z) + \arg(1-z)\,\log|z|.$$

Let $w = e^{i\pi/3}$, the shape of the regular ideal tetrahedron, with $V_0 = D(w) = 1.0149416064\ldots$, the largest possible. Define the **vacuum disk**

$$G(z) = \frac{z-w}{z-\bar{w}}.$$

Proposition 1 (vacuum, rim, mirror, racket).

(a) $G(w) = 0$: the regular tetrahedron is the vacuum point.

(b) Real z (a flat tetrahedron, $D = 0$) gives $|G| = 1$: flat pieces lie on the rim.

(c) The mirror shape $\bar{z}$ gives $|G(\bar{z})| = 1/|G(z)|$: mirrors lie outside the disk.

(d) The axis switch $T(z) = 1/(1-z)$, which relabels the tetrahedron's edges ($z \to z' \to z''$), acts on G as an exact rotation, $G(Tz) = e^{2\pi i/3}\,G(z)$, while $D(Tz) = D(z)$.

Proof.

- (a) is immediate.
- (b): for real z, $|z-w| = |z-\bar{w}|$, because $\bar{w}$ is the reflection of w in the real line.
- (c): $G(\bar{z}) = (\bar{z}-w)/(\bar{z}-\bar{w}) = \overline{(z-\bar{w})/(z-w)} = \overline{G(z)}^{-1}$.
- (d), step 1: T is a Möbius map of order 3 whose fixed points are the roots of $z^2 - z + 1 = 0$, namely w and $\bar{w}$.
- (d), step 2: a Möbius map with fixed points $w, \bar{w}$ becomes, in the coordinate G that sends $w \mapsto 0$ and $\bar{w} \mapsto \infty$, multiplication by its multiplier at w.
- (d), step 3: that multiplier is $T'(w) = 1/(1-w)^2 = 1/e^{-2i\pi/3} = e^{2\pi i/3}$.
- (d), step 4: the invariance $D(z) = D(1/(1-z)) = D(1-1/z)$ is the classical symmetry of the Bloch–Wigner function. □

Checked numerically: for $z = 0.3 + 1.1i$ the three labels sit at radius 0.1557583601, at 34.715°, 154.715° and −85.285°, with identical $D = 0.9832288623$. *Status:* (a)–(c) are elementary; (d) is a new reading of a classical symmetry; the racket interpretation is ours.

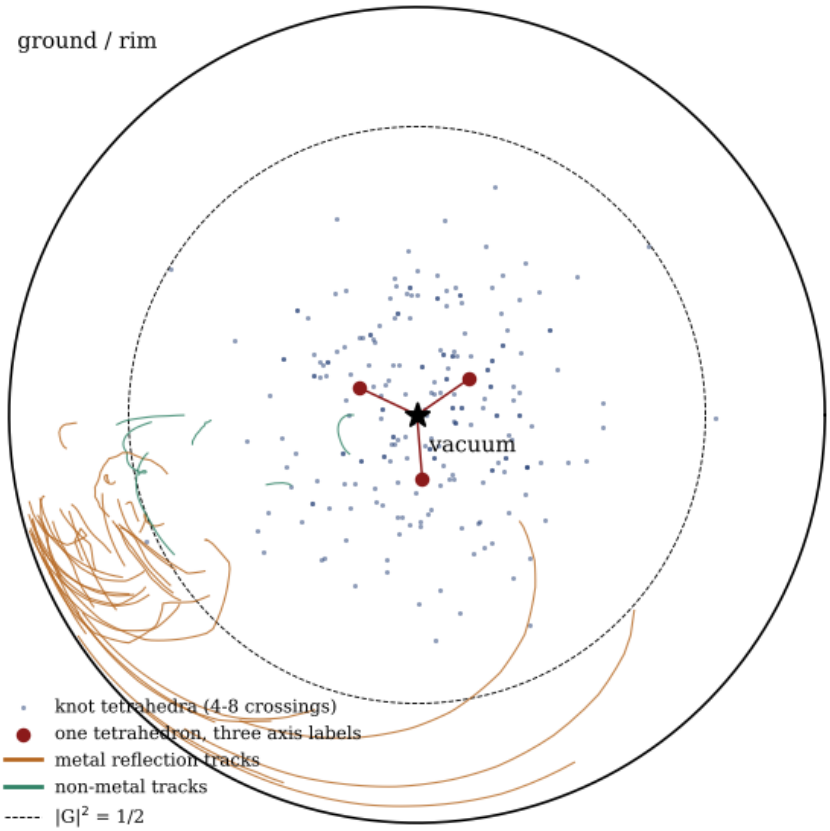


Figure 1: The vacuum disk. Knot tetrahedra (blue, 4–8 crossings) crowd around the vacuum point. One tetrahedron's three axis labels (red) are 120° apart at one radius. The reflection tracks of the elements (Section 9) lie on the same disk: metals (orange) near the rim, non-metals (green) further in. The dashed circle is $|G|^2 = 1/2$.

3. Decay: the vacuum bowl

Theorem 2 (bowl series). Near the vacuum point,

$$D(z) = V_0 - \frac{3\sqrt{3}}{4}|G|^2 + \frac{\sqrt{3}}{8}\left(G^3 + \bar{G}^3\right) + \frac{3\sqrt{3}}{16}|G|^4 + O(|G|^5).$$

Status: derived symbolically (sympy expansion of D in the coordinate G) and confirmed numerically. $(V_0 - D)/|G|^2 \to 1.2990$ against $3\sqrt{3}/4 = 1.29904$, and the residual scales as $|G|^5$.

What each term means.

- The quadratic term is a round bowl: volume drops with squared distance from the vacuum.
- The cubic term is the racket written exactly. It has the three-fold symmetry of Proposition 1(d) and nothing else.
- The quartic term is the next radial correction.

The knot law (nothing fitted). A knot's volume is the sum of its pieces,

$$\mathrm{Vol}(K) = \sum_j D(z_j) \approx \sum_j \left[V_0 - \tfrac{3\sqrt{3}}{4}|G_j|^2 + \ldots\right].$$

Law	Knots	Median error	90th pct	Worst
pieces all at full volume, nV_0	243	10.9%		
bowl, quadratic only	243	0.27%	0.96%	13% (10_1)
bowl, quadratic only	12,954	0.21%	0.62%	19.9% (12a_803)
bowl + cubic	12,954	0.39%	0.90%	27%
bowl + cubic + quartic	**12,954**	**0.076%**	**0.31%**	**11.4% (11a_247)**
opposite: hyperbolic distance	12,954	1.3%	3.2%	>200%

- **The cubic needs the quartic.** The racket term alone over-corrects; it only helps once the quartic term is included. With both, 12a_803 improves from −19.9% to −4.5%.
- **Most pieces sit near the vacuum.** Real knots keep most of their pieces close in: 1,391 of 2,777 tetrahedra lie within hyperbolic distance 0.5. That is why the bowl works.
- **Reading.** *A knot's volume is its pieces at full volume, minus how far each one sits from the vacuum.* This is the decay book.

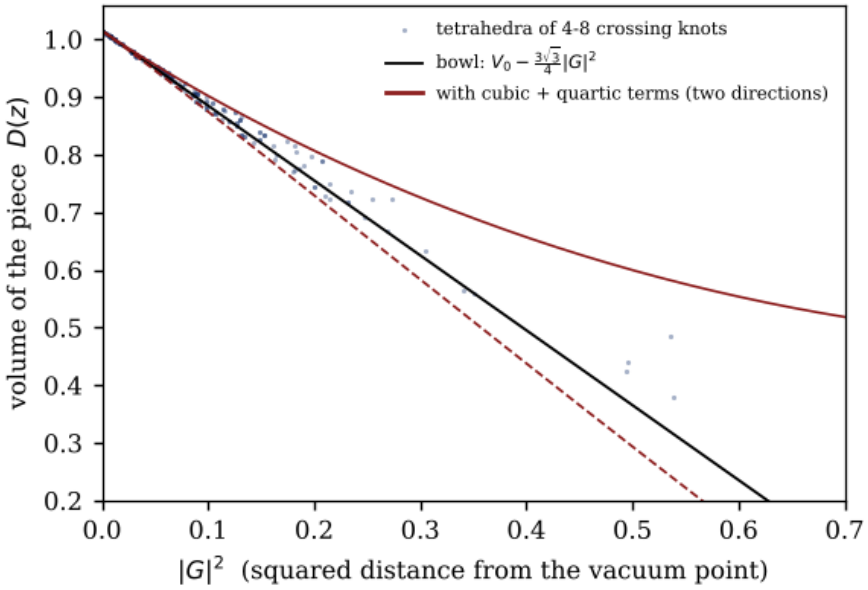


Figure 2: The vacuum bowl. Each dot is one tetrahedron. The black line is the quadratic bowl; the red curves add the cubic and quartic terms along the two extreme directions of the three-fold term.

4. Every geometry, and the one nearest the vacuum

A knot's gluing equations (edges plus meridian completeness) have more than one solution. Besides the true hyperbolic structure there are its mirror, flat solutions and Galois-conjugate ones. Each is a different way of putting a geometry on the knot.

Method. All solutions were found by damped Newton iteration from 3,000 random starts.

Results (31 hyperbolic knots up to 8 crossings, 291 solutions):

Check	Result
true geometry = smallest $\sum \lvert G_j\rvert^2$ (nearest the vacuum)	31 of 31
true geometry = largest volume	31 of 31 (known: Francaviglia; Cho)
mirrors outside the disk	always
all-real solutions	$\sum \lvert G_j\rvert^2 = n$ exactly (on the rim, Prop. 1b)
within-knot order: $\sum \lvert G\rvert^2$ against volume	median Spearman -0.905
ladder breaks between two fully-inside solutions	none of 136 inversions

Where the ladder breaks.

- Every break involves a flat solution or a mixed one, with pieces on both sides of the rim.
- Counting outside pieces as mirrored, $\varepsilon_j[V_0 - k|\tilde{G}_j|^2]$ with $\tilde{G} = 1/\bar{G}$ and $\varepsilon = -1$ outside, restores the order: median Spearman 0.995.
- Example (6_1): $\sum |G|^2 = 0.74, 5.1, 28.9, 47.3$ against volumes $3.164, 1.415, -1.415, -3.164$.

Reading: **the knot's geometry is the root nearest the vacuum.** It is known that the geometric solution maximises volume; that it is the one nearest the vacuum, with the bowl law as the reason, is our reading.

5. Direction: the twist lives in integers

The complex volume $\mathrm{Vol} + i\,\mathrm{CS}$ has a second half, the Chern–Simons invariant.

Null result first. In the experiments it was *not* found in the angles of the pieces: the summed three-fold angle of G against CS gives $\rho = 0.02$.

Neumann's formula. Give each tetrahedron log-parameters

$$w_0 = \log z + p\pi i, \quad w_1 = \log z' + q\pi i, \quad w_2 = -w_0 - w_1, \qquad z' = \tfrac{1}{1-z},$$

with integers p, q (the windings) chosen so that every edge and both cusp curves have total log-parameter 0 (a flattening; Zickert, Def. 6.1). Then

$$i(\mathrm{Vol} + i\,\mathrm{CS}) = \sum_j \varepsilon_j\, R(z_j; p_j, q_j),$$

$$R(z; p, q) = \mathrm{Li}_2(z) + \tfrac{1}{2}\log z \log(1-z) + \tfrac{\pi i}{2}(q \log z + p \log(1-z)) - \tfrac{\pi^2}{6}.$$

Results.

- The integer system was solved exactly (PARI `matsolvemod`) for 79 hyperbolic knots of 4–9 crossings.
- **Volume** needs no integers: it matches SnapPy to 5×10^{-14}.
- **Chern–Simons**, with the windings, first matched only up to $\pm\pi^2/6$ on 54 knots.
  - The cause: R is not invariant under relabelling a tetrahedron. $R(z'; q, r) - R(z; p, q) = (1 + 3q)\pi^2/6$.
  - The fix: read every tetrahedron in a *branching* order (a vertex order with order-preserving gluings, found on all 79). Odd permutations invert the shape and negate the integers ($\varepsilon = -1$).
  - With that, CS is exact in **454 of 454** runs (79 knots × up to 6 branchings), to 2.7×10^{-14}, modulo SnapPy's period $\pi^2/2$.
- **The windings are essential.** With them dropped, the continuous part alone matches 1 of 79.

Reading. The two books separate in the knot's own geometry. *Decay (volume) is continuous: distance from the vacuum. Direction (twist) is integer: how many half-turns each piece's phase winds.* This is the IQGT split $Q = g - \frac{i}{2}\Omega$ again, and it is the phase winding the author's work began with.

Status: reproduction of Neumann's theorem; the branching fix is standard. The two-book reading is ours.

6. Our polynomial, read off the drawing

The potential. Cho–Kim–Kim (2013) define a potential of a knot *diagram*.

- One variable z_k per side; each crossing with sides (a, b, c, d) adds

$$\mathrm{Li}_2\left(\tfrac{z_b}{z_a}\right) - \mathrm{Li}_2\left(\tfrac{z_b}{z_c}\right) + \mathrm{Li}_2\left(\tfrac{z_d}{z_c}\right) - \mathrm{Li}_2\left(\tfrac{z_d}{z_a}\right).$$

- The hyperbolicity equations $\exp(z_k\,\partial V/\partial z_k) = 1$ are products of $(1 - \text{ratio})^{\pm 1}$, so no logarithm branch has to be chosen.
- The modified potential $V_0 = V - \sum_k (z_k \partial_k V) \log z_k$ gives $i(\mathrm{Vol} + i\,\mathrm{CS})$ at the geometric solution (CKK Thm 1.2).
- This is, literally, *the volume of the knot potential*.

Reading any drawing.

- The placement of the four sides at a crossing appears in the source only as a figure, so all 64 placements were tested against SnapPy.
- One rule survives, up to the term's symmetry ($a \leftrightarrow c,\ b \leftrightarrow d$): (b, d) is the under-strand pair of the PD crossing $X[i, j, k, l]$, the same for both signs.
- A kinked crossing $X[i, j, j, l]$ is singular and must be removed.

The finite object.

- The solutions come in continuous families: no single ratio or cross-ratio is fixed along them.
- Only V_0 is constant on each family. So the drawing gives one complex number per family, that is, per boundary-parabolic geometry.
- Its imaginary part is the **decay** (volume of that geometry); its real part is the **direction** ($-\mathrm{CS} \bmod \pi^2$).
- We define the knot's **decay–direction polynomial**

$$P_K(x) = \prod_{\rho} \left(x - e^{\lambda V_0(\rho)}\right),$$

one root per geometry, with λ chosen to map the π^2 period to a full turn. Its roots carry the two books directly.

Results.

Test	Result
top root = SnapPy complex volume (vol to 10^{-6}, CS to 10^{-5})	79 of 79 hyperbolic knots, 4–9 crossings
same full spectrum from three different drawings (one with 2 extra crossings)	13 of 13 knots
spectrum volumes = the ladder of Section 4	exact

The search is a budget question, not a convergence question: on hard knots about one random start in a thousand reaches the geometric family.

Knot	Every geometry as (decay, direction)	SnapPy
4_1	(2.02988, 0); mirror	(2.02988, 0)
5_2	(2.82812, −3.0241), (0, 1.1135); mirror	(2.82812, −3.0241)
6_1	(3.16396, 3.0789), (1.4151, 0.211); mirrors	(3.16396, 3.0789)
6_2	(4.40083, −3.997), (1.53058, 0.3291), (0, 2.4011); mirrors	(4.40083, −3.997)

Knot	Every geometry as (decay, direction)	SnapPy
6_3	(5.69302, 0), (0.9243, ±1.8906); mirrors	(5.69302, 0)

Caveat: SnapPy's named manifold is the mirror of the KnotInfo drawing for 42 of the 79 knots. Signed comparisons must use the exterior of the drawing itself.

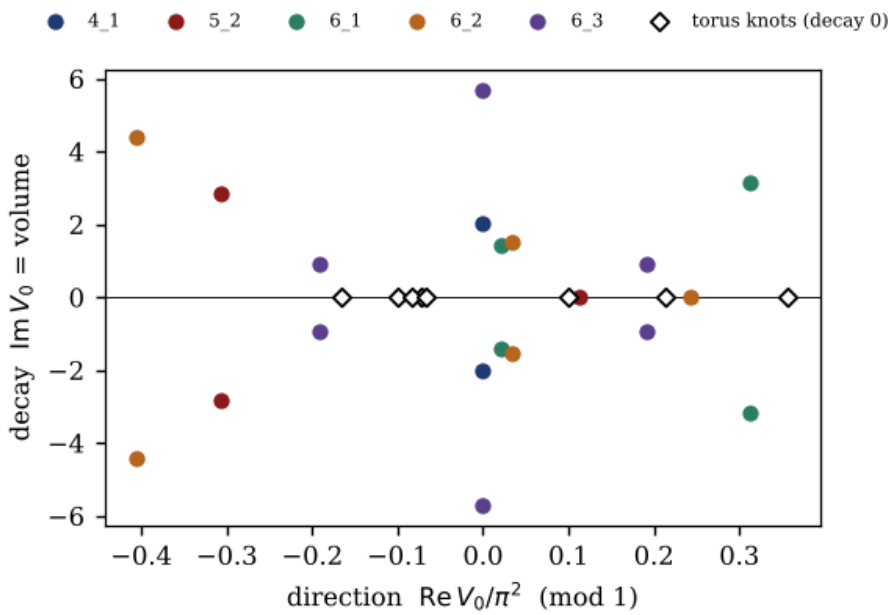


Figure 3: Our polynomial's roots, read off drawings. Each knot gives a ladder of geometries with a decay (volume) and a direction (twist). Torus knots (diamonds) sit on the zero-decay line at rational directions.

Torus knots: pure direction. On 3_1, 5_1, 7_1, 8_19 and 10_124 every valid root has decay 0 (to 10^{-15}), and every direction is an exact rational multiple of π^2:

Knot	Directions (fraction of π^2, mod 1)
3_1 = T(2,3)	5/6
5_1 = T(2,5)	1/10, 9/10
7_1 = T(2,7)	3/14, 5/14, 13/14
8_19 = T(3,4)	11/12
10_124 = T(3,5)	14/15

- All values fit $-k^2/(pq)$ mod 1.
- *Status:* reproduction. These are the Chern–Simons values of torus-knot representations (Kirk–Klassen; computed explicitly by Dubois–Kashaev and Murakami).
- The known count of components, $(a-1)(b-1)/2$, matches $T(2,q)$ exactly. It predicts 3 directions for 8_19 and 4 for 10_124, of which our search found 1 each: a clean target for the next run.

7. Light strings: Jones and HOMFLY

The Jones string.

- Let $t = e^{i\theta}$ run once around the circle, as colour runs from violet to red, and follow the Jones polynomial $V(t)$. Because $V(1) = 1$, the string leaves one point and returns to it.
- **Climb** is the Mahler measure (mean of $\log|V|$).
- **Turn** is the signed area swept, divided by the mean of $|V|^2$.

On 2,970 hyperbolic knots up to 12 crossings (and 9,985 at 13 crossings in an independent run):

Quantity	Against	Result
climb	volume	$\rho = 0.854$ (independent run: 0.8542)
climb, after log det	volume left over	$\rho = 0.50$; R^2: 0.708 (log det alone) $\rightarrow$ 0.810
turn (swept area)	|signature|	$\rho = 0.83$
turn counted as roots inside the circle	volume or signature	nothing
any of these, and the angles of Alexander or Jones zeros	Chern–Simons	no correlation found (bears on an open question of Friedl–Jackson)

Prior art: the Mahler measure of Jones against volume is known (Champanerkar–Kofman), as is the determinant (Dunfield; Champanerkar–Kofman–Purcell). Combining them, and the turn–signature split, were not found in those sources.

HOMFLY along its light lines.

- On the plain two-phase torus $|v| = |z| = 1$, HOMFLY carries nothing ($\rho = 0.036$).
- The opposite test put light where knot theory lives: the sl(N) lines $v = t^{-N/2}$, $z = t^{1/2} - t^{-1/2}$. N = 0 is Alexander and N = 2 is Jones; the Jones slice reproduces the Jones climb to 3×10^{-13}.
- On 12,955 knots, the climb carries volume at $\rho = 0.60$ (N = 0), 0.86 (N = 2), and 0.78–0.86 for N = 3..8.
- Each higher line adds a slice the Jones line misses. All lines with log det reach $R^2 = 0.838$, against 0.795 for log det + Jones.
- The surface the lines sweep gives $\rho = 0.864$.
- VOTKP's first meaning, a HOMFLY and coloured-Jones mix mapping volume, is borne out on these lines. *Prior art to check:* sl(N) volume conjectures.

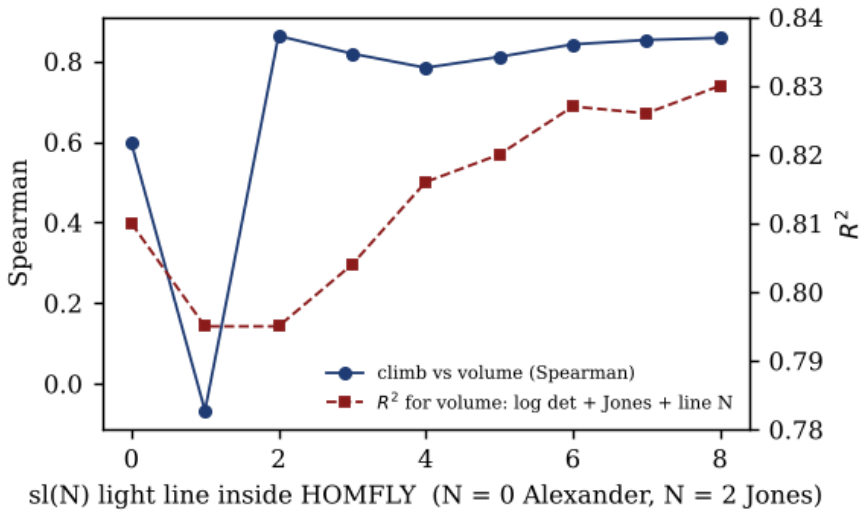


Figure 4: HOMFLY read with light. Climb along each sl(N) line against volume (blue), and the share of volume explained when that line is added to log det and the Jones climb (red).

The circuit polynomial (first attempt).

- *Construction.* Shade the diagram. Shaded regions are nodes and crossings are branches; a right-handed crossing has admittance z, a left-handed one 1. $Q(z)$ counts spanning trees by right-handed branches.
- $Q(1) = \det K$ on 1,846 of 1,846 alternating knots.
- Q is unchanged on 180 of 180 reduced alternating redrawings. *Proof sketch:* a flype acts on the Tait graph as a Whitney twist, which keeps every spanning tree, and keeps every crossing's handedness; by Tait's flyping theorem Q is then an invariant of alternating knots.
- It is coarse: knots with all crossings of one hand give a constant.
- Its lesson: on the diagram alone, the only fully invariant circuit weights lead back to the Kauffman bracket, that is, to Jones. A better polynomial needs the geometry the diagram lacks, which is why Section 6 reads the potential and its geometries.

8. Lessons from surfaceology

- **What Figueiredo's group found.** Carolina Figueiredo, with Arkani-Hamed, Cao, Dong and He, found hidden zeros of $\mathrm{Tr}(\phi^3)$ amplitudes and the one kinematic shift that keeps them. That shift turns a theory with only denominators into pions and gluons, with numerators.
- **Reproduced here:**
  - the 6-point amplitude vanishes exactly on its hidden-zero locus;
  - the shifted 4-point amplitude has leading term $-(X_{13} + X_{24})$;
  - the shifted 6-point amplitude equals the NLSM amplitude (ratio 1 at three random points).
- **Her u-variables are cross-ratios.** They satisfy $u_C + \prod u_D = 1$ (checked to 4×10^{-16}) and are unchanged by Möbius maps. They are the same kind of number as the tetrahedron shapes, and Möbius maps are the Smith chart's own moves.
- **Lessons taken:**
  - find the zeros first;
  - a denominator-only object, like the circuit polynomial, needs a shift to gain numerators.

9. Colour: the elements on the same disk

Reflection tracks. The reflection $\Gamma = (1 - N)/(1 + N)$ of each element across the visible band traces a track on the Smith disk, using 41 elements from refractiveindex.info.

What showed up.

- **The Schottky line.** $|\Gamma|^2 = 1/2$, averaged over the visible, separates metals (35 of 35 above) from non-metals (0 of 6 above).
- **Hidden phase.** Bulk colour hides the phase of Γ, and a thin film writes it back. Over 39 metal pairs that the eye cannot tell apart in bulk, the phase gap predicts the colour gap under a film at $\rho = 0.64$. Pairs riding the rim under-read ($\rho = -0.52$, $p = 0.0007$): near ground the phase is present but pinned.
- **One handedness.** Every element's track winds the same way (38 of 41, the other three about 0). Every causal model medium does the same, through absorption lines too: causality and passivity, in the family of Foster's reactance theorem. *Status:* recognised physics.
- **The space between.** Molecular solids crossing the 2/3 vacuum-mix line under squeeze rank their measured metallization pressures at $\rho = 0.89$ (Te, P, Se, I, Br, S). Network solids (Si, Ge, B) already sit on the line and need a flip, a change of structure, not a squeeze.
- **The film winding is an integer index of the substrate.** It is exact to 10^{-15} and grows by one per 247 nm of film. It jumps across the circle $|r_{12}| = |r_{01}|$; selenium is the one element inside it.
  - Lightness follows the bulk reflection (continuous).
  - Chroma follows the integer order.
  - Hue follows the leftover phase.
  - The shape matches volume beside windings; the physics is Newton's colour orders.
- **Knots rendered as colours.** Jones strings rendered as colours carry volume through lightness ($\rho = 0.96$; R^2 0.707 $\to$ 0.878), but that colour is a compression of $|V|$.

What is not yet joined. The knot pieces fill the inside of the disk; the metals sit near the rim (Figure 1). The bowl region is barely reached by the elements. Clear materials (oxides, salts, glasses) live inside the disk and are

the next colour test.

10. The electron

The electron's group is the chain's group.

- An electron's spin lives in $\mathbb{C}^2$, turned and boosted by $\mathrm{SL}(2,\mathbb{C})$, the double cover of the Lorentz group.
- Dropping the sign gives $\mathrm{PSL}(2,\mathbb{C})$. That group is at once the Möbius maps (the Smith chart's moves) and the symmetries of hyperbolic 3-space, where the knot's volume lives.
- Every geometry of Section 4 is a representation of the knot group into this group.
- So the chain *light* → *electron* → *knot* runs through one group. *Status:* standard mathematics; its use as the spine of the chain is ours.

The electron decides where matter sits on the disk. Free electrons push reflection to the rim (ground); bound electrons keep it near the vacuum. The metal/non-metal line and the 2/3 space-between line are the electron's choice to conduct.

The double cover and the last open piece of the twist (conjecture, untested).

- The twist of Section 5 is exact modulo $\pi^2/2$; whether it is exact modulo π^2 is open.
- Lifting a knot's geometry from PSL to $\mathrm{SL}(2,\mathbb{C})$ is a choice of spin structure: the electron's 720° return, the -1 of Dirac Time §5.2.
- The conjecture: the remaining $\pi^2/2$ is that choice, so fixing the spin lift closes the direction book.

11. Knots that build themselves

Molecular knots.

- Leigh's group braided the 8_19 knot from three strands around metal ions (2017).
- Micheletti's self-assembly simulations had predicted it, and found knot preference unrelated to crossing count.
- Their 2018 survey names the privileged knots 3_1, 4_1, 5_1 and 8_19, plus two new ones, 10_124 and 15n_41185.

Prediction, made before the lookup: the knots that self-assemble are those that make no space, the torus knots.

Checks.

Knot	Torus knot	Hyperbolic?	Cyclic covers (2–6) match the braid-built torus knot?
10_124	T(3,5)	no	yes
15n_41185	T(4,5)	no	yes

T(4,5) is also the only non-alternating 15-crossing torus knot.

Result. Five of the six privileged knots are torus knots: zero decay, pure direction. The exception, 4_1, has the smallest volume of any hyperbolic knot. *Status:* the fingerprint match is strong but not a formal identification; the law is untested beyond these six.

12. Ledger: what is ours

Item	Status
Volume $= \sum D(z_j)$; CS from flattenings; torus-knot CS; max volume picks the geometry; potential of a diagram	REPRODUCTION (Thurston, Milnor, Neumann, Zickert, Kirk–Klassen, Francaviglia, Cho, Cho–Kim–Kim)

Item	Status
Vacuum disk G; racket = exact 120° rotation; rim = flat; mirror outside	NEW READING (elementary proofs, Prop. 1)
Bowl series and the no-fit knot law (median 0.076%, 12,954 knots)	NEW RESULT in our frame (the series is a Taylor expansion; the law and its error budget are ours)
Nearest-the-vacuum rule; orientation-aware ladder	NEW READING with new tests (31/31; $\rho = 0.995$)
Placement rule for reading CKK off any PD drawing; 79/79; 13/13 diagram independence	NEW RESULT (method)
Decay–direction polynomial P_K	NEW DEFINITION; arithmetic of its coefficients open
Jones climb + log det; turn ↔ signature; HOMFLY sl(N) lines carry extra volume	NEW RESULTS (empirical)
Circuit polynomial Q, flype invariance	NEW (prior-art check pending: clock states, Crowell)
Self-assembly ↔ zero decay	NEW OBSERVATION, untested as a law
Electron spin lift closes the twist	CONJECTURE

13. Open problems

1. The missing torus roots: 2 for 8_19, 3 for 10_124, using the known (k, l) indexing.
2. The spin lift: exactness of CS modulo π^2.
3. The arithmetic of P_K: its coefficients through the trace field, with LMFDB labels.
4. Colour inside the disk: dielectrics and the bowl.
5. Prior art on the circuit polynomial and on the sl(N) climb.
6. The 61 silent double walls of VOTKP v2.9 Table 6, in infinite-order knots, not covered by theory.

References

- J. Beasley, *Intrinsic Quantum Geometric Tensor*, v7.3.1, GitHub dicipler-pixel/iqgt-grassmannian-lean.
- J. Beasley, *Dirac Time of the Gigantefermion*, v3.1, Zenodo.
- J. Beasley, *Elemental Peeling (VOTKP)*, v2.9, Zenodo.
- W. Thurston, *The Geometry and Topology of Three-Manifolds*, lecture notes, Princeton.
- W. D. Neumann, Extended Bloch group and the Cheeger–Chern–Simons class, *Geom. Topol.* 8 (2004) 413–474; arXiv:math/0307092.
- C. Zickert, The volume and Chern–Simons invariant of a representation, arXiv:0710.2049.
- J. Cho, H. Kim, S. Kim, Optimistic limits of Kashaev invariants and complex volumes of hyperbolic links, arXiv:1301.6219.
- J. Cho, J. Murakami, Optimistic limits of the colored Jones polynomials, *J. Korean Math. Soc.* 50 (2013) 641–693; arXiv:1009.3137.
- J. Cho, Yokota theory, the invariant trace fields of hyperbolic knots and the Borel regulator map, arXiv:1005.3094.
- J. Dubois, R. Kashaev, On the asymptotic expansion of the colored Jones polynomial for torus knots, arXiv:math/0510607.
- H. Murakami, An introduction to the volume conjecture and its generalizations, arXiv:0802.0039.
- State integrals for the quantized SL2(C) Chern–Simons invariant, arXiv:2601.05136 (2026).
- A. Champanerkar, I. Kofman, On the Mahler measure of Jones polynomials under twisting, *Algebr. Geom. Topol.* 5 (2005); arXiv:math/0404236.
- A. Champanerkar, I. Kofman, J. Purcell, Geometrically and diagrammatically maximal knots, *J. London Math. Soc.* 94 (2016) 883–908.
- S. Friedl, N. Jackson, Approximations to the volume of hyperbolic knots, arXiv:1102.3742.
- N. Arkani-Hamed, Q. Cao, J. Dong, C. Figueiredo, S. He, Hidden zeros for particle/string amplitudes and the unity of colored scalars, pions and gluons, *JHEP* 10 (2024) 231; arXiv:2312.16282.
- N. Arkani-Hamed, Q. Cao, J. Dong, C. Figueiredo, S. He, NLSM in $\mathrm{Tr}(\phi^3)$, arXiv:2401.05483.
- N. Arkani-Hamed, C. Figueiredo, Circles and Triangles, the NLSM and $\mathrm{Tr}(\Phi^3)$, arXiv:2403.04826.
- J. J. Danon et al., Braiding a molecular knot with eight crossings, *Science* 355 (2017) 159–162.

- M. Marenda, E. Orlandini, C. Micheletti, Discovering privileged topologies of molecular knots with self-assembling models, *Nat. Commun.* (2018).
- R. Montgomery, How much does the rigid body rotate? A Berry's phase from the 18th century, *Am. J. Phys.* 59 (1991) 394.
- KnotInfo (Livingston, Moore); SnapPy (Culler, Dunfield, Goerner, Weeks); refractiveindex.info (Polyanskiy).

Appendix C. Code

The scripts behind each result are listed below in full. The archive holds every script from the session (Appendix D lists them), with outputs and notes.

shape_disk_01.py

```
# SCRIPT: SHAPE-DISK-01
# What VOTKP seeks, on the Smith disk. Each hyperbolic knot complement is glued from ideal tetrahedra;
 ↪  each has a shape z
# (a cross-ratio, Im z > 0) and contributes Bloch-Wigner volume D(z). Put every shape on a Smith disk
 ↪  centred on the
# regular tetrahedron w = e^{i pi/3}:  G = (z - w)/(z - conj(w)).
# Why this tests the picture:
#  (1) the axis switches z -> 1/(1-z) -> 1-1/z should become exact 120-degree turns of G about the
 ↪  centre, with D unchanged
#      (racket: axis switches, the conserved quantity stays);
#  (2) the regular tetrahedron (max volume 1.01494) sits at the vacuum point G = 0 and flat ones (z real)
 ↪  on the rim;
#  (3) the climb book (hyperbolic distance of G from the centre) should then carry each tetrahedron's
 ↪  volume, and the
#      knot's volume should be read from how close its pieces sit to the vacuum.
# Also: the turn book (angle of G) against the Chern-Simons half, per knot (snappy complex volume).
import numpy as np, snappy, warnings, database_knotinfo as d, re
from mpmath import polylog
from scipy.stats import spearmanr
warnings.filterwarnings('ignore')
w=np.exp(1j*np.pi/3)
def D(z): return float(np.imag(complex(polylog(2,z))))+np.angle(1-z)*np.log(abs(z))
def G(z): return (z-w)/(z-np.conj(w))
# (1),(2)
z=0.3+1.1j; zs=[z,1/(1-z),1-1/z]
print('(1) D on the three axis labels:',[round(D(x),10) for x in zs])
print('    angles of G (deg):',[round(np.degrees(np.angle(G(x))),6) for x in zs],'
 ↪  radii:',[round(abs(G(x)),10) for x in zs])
print('(2) regular tetrahedron: G =',abs(G(w)),' D =',round(D(w),6),' ; nearly flat z=2+1e-6i: |G|
 ↪  =',round(abs(G(2+1e-6j)),6))
# (3) per tetrahedron and per knot
pts=[]; knots=[]
names=[r['name'] for r in d.link_list() if re.match(r'^\d+_\d+$',r['name'] or '') and
 ↪  4<=int(r['name'].split('_')[0])<=10]
for nm in names:
    try:
        M=snappy.Manifold(nm)
        if M.solution_type()!='all tetrahedra positively oriented': continue
        sh=[complex(s) for s in M.tetrahedra_shapes('rect')]; cv=complex(M.complex_volume())
    except Exception: continue
    g=np.array([G(s) for s in sh]); dist=2*np.arctanh(np.abs(g)); dv=np.array([D(s) for s in sh])
    for a,b in zip(dist,dv): pts.append((a,b))
```

```
    knots.append((nm,cv.real,cv.imag,len(sh),dist,dv,g))
P=np.array(pts)
print('(3) tetrahedra:',len(P),' from',len(knots),'knots')
print('    per tetrahedron: distance from vacuum vs volume  rho=%.3f'%spearmanr(P[:,0],P[:,1])[0])
for lo,hi in [(0,0.5),(0.5,1),(1,1.5),(1.5,2.5),(2.5,9)]:
    m=(P[:,0]>=lo)&(P[:,0]<hi)
    if m.sum(): print('      distance %.1f-%.1f: n=%4d  volume %.3f..%.3f (mean
     ↪  %.3f)'%(lo,hi,m.sum(),P[m,1].min(),P[m,1].max(),P[m,1].mean()))
vol=np.array([k[1] for k in knots]); cs=np.array([k[2] for k in knots]); nt=np.array([k[3] for k in
 ↪  knots])
fill=np.array([np.sum(np.cos(np.arctan(0.5*k[4]))**2) for k in knots])     # placeholder radial weight,
 ↪  tested below
rad=np.array([np.sum(1-np.abs(k[6])**2) for k in knots])                    # how much of each piece is
 ↪  'vacuum'
print('    per knot: volume vs number of pieces rho=%.3f ; vs summed (1-|G|^2)
 ↪  rho=%.3f'%(spearmanr(nt,vol)[0],spearmanr(rad,vol)[0]))
print('    volume / summed (1-|G|^2): mean %.4f  spread (std/mean)
 ↪  %.3f'%(np.mean(vol/rad),np.std(vol/rad)/np.mean(vol/rad)))
turn=np.array([np.sum(np.angle(k[6]**3)) for k in knots])                   # 3-fold-invariant angle of
 ↪  each piece
print('(4) turn: summed 3-fold angle vs CS  rho=%.3f ; |.| vs |CS|
 ↪  rho=%.3f'%(spearmanr(turn,cs)[0],spearmanr(abs(turn),abs(cs))[0]))
# (5) the vacuum-well law at knot level: Vol ~ sum over pieces of [V0 - (3 sqrt3/4)|G|^2]  (no fitting)
V0=1.0149416064096536; k2=3*np.sqrt(3)/4
pred=np.array([np.sum(V0-k2*np.abs(k[6])**2) for k in knots])
err=(pred-vol)/vol
print('(5) well law, no fit: median |error| %.2f%%, 90th pct %.2f%%, worst %.2f%% (%s)'%(100*np.median(
 ↪  abs(err)),100*np.percentile(abs(err),90),100*abs(err).max(),knots[int(np.argmax(abs(err)))][0]))
print('    pieces at full volume alone (n*V0): median |error|
 ↪  %.2f%%'%(100*np.median(abs(nt*V0-vol)/vol)))
```

shape_disk_02.py

```
# SCRIPT: SHAPE-DISK-02
# How exact is "volume = how close each piece sits to the vacuum point"? Per tetrahedron, compare D(z)
 ↪  with
# V0*(1-|G|^2)^p (V0 = 1.01494, regular tetrahedron) across the whole disk, then the best power p, then
 ↪  the knot totals.
import numpy as np
from mpmath import polylog
w=np.exp(1j*np.pi/3); V0=1.0149416064096536
def D(z): return float(np.imag(complex(polylog(2,z))))+np.angle(1-z)*np.log(abs(z))
rng=np.random.default_rng(0)
g=np.sqrt(rng.uniform(0,1,4000))*np.exp(1j*rng.uniform(0,2*np.pi,4000))      # uniform over the disk
z=(w-np.conj(w)*g)/(1-g)                                                  # inverse of G = (z-w)/(z-conj w)
dv=np.array([D(x) for x in z]); f=1-np.abs(g)**2
for r0,r1 in [(0,.3),(.3,.6),(.6,.8),(.8,.95),(.95,1)]:
    m=(abs(g)>=r0)&(abs(g)<r1); q=dv[m]/(V0*f[m])
    print('|G| %.2f-%.2f  D/(V0(1-|G|^2)) = %.4f .. %.4f'%(r0,r1,q.min(),q.max()))
m=abs(g)<0.9; A=np.column_stack([np.log(f[m]),np.ones(m.sum())]);
 ↪  c,*_=np.linalg.lstsq(A,np.log(dv[m]),rcond=None)
print('best power law over |G|<0.9: D ~ %.4f (1-|G|^2)^%.4f'%(np.exp(c[1]),c[0]))
# second-order: D near the centre in terms of |G|^2 (exact Taylor check)
for e in (1e-2,1e-3):
    gg=e*np.exp(1j*np.linspace(0,2*np.pi,12,endpoint=False)); zz=(w-np.conj(w)*gg)/(1-gg)
```

```
    print('near centre |G|=%g: (V0-D)/|G|^2 over 12 angles = %.6f .. %.6f'%(e,min((V0-D(x))/e**2 for x in
     ↪  zz),max((V0-D(x))/e**2 for x in zz)))
print('compare: V0 = %.6f, 3*sqrt(3)/2 = %.6f, sqrt(3) = %.6f'%(V0,3*np.sqrt(3)/2,np.sqrt(3)))
```

roots_disk_02.py

```
# SCRIPT: ROOTS-DISK-02 (wider damped search, 3000 starts)
# All solutions of a knot's gluing equations (edges + meridian completeness) = every way to put a
 ↪  hyperbolic-type
# geometry on the knot: the true one, its mirror, flat (real) ones, and other Galois-conjugate ones.
 ↪  These are the roots
# of the knot's shape polynomial. On the Smith disk centred at the regular tetrahedron, G = (z - w)/(z -
 ↪  conj w):
# prediction (from the vacuum-well law): the true geometry is the solution whose pieces sit nearest the
 ↪  vacuum
# (smallest sum |G|^2); mirrors sit outside the disk; flat solutions on the rim. Compare: max volume
 ↪  picks the true one
# (Cho; Francaviglia). Each solution's 'decay' = its volume (sum D), and the test asks whether
 ↪  nearest-vacuum agrees.
import numpy as np, snappy, warnings, re, database_knotinfo as d
from mpmath import polylog
warnings.filterwarnings('ignore')
w=np.exp(1j*np.pi/3)
def D(z): return float(np.imag(complex(polylog(2,z))))+np.angle(1-z)*np.log(abs(z))
def solve_all(M,starts=3000,seed=0):
    g=M.gluing_equations('rect'); A=np.array([r[0] for r in g],float); B=np.array([r[1] for r in
 ↪  g],float); c=np.array([r[2] for r in g],complex)
    n=A.shape[1]; rows=list(range(n-1))+[n]                  # n-1 edges + meridian
    A,B,c=A[rows],B[rows],c[rows]
    def F(z): return np.exp(A@np.log(z)+B@np.log(1-z))-c       # principal logs only enter through exp:
     ↪  branch-free
    def J(z):
        f=np.exp(A@np.log(z)+B@np.log(1-z))
        return f[:,None]*(A/z[None,:]-B/(1-z)[None,:])
    rng=np.random.default_rng(seed); sols=[]
    for _ in range(starts):
        sc=rng.choice([0.3,1,3,10]); z=rng.normal(0.5,sc,n)+1j*rng.normal(0,sc,n)
        for it in range(120):
            try: dz=np.linalg.solve(J(z),-F(z))
            except np.linalg.LinAlgError: break
            st=min(1.0,0.5/max(1e-12,np.max(abs(dz))/max(1,np.max(abs(z)))))); z=z+st*dz
            if np.max(abs(dz))<1e-13: break
        if np.all(np.isfinite(z)) and np.max(abs(F(z)))<1e-9 and np.min(abs(z))>1e-6 and
         ↪  np.min(abs(1-z))>1e-6:
            if not any(np.max(abs(z-s))<1e-7 for s in sols): sols.append(z)
    return sols
names=[r['name'] for r in d.link_list() if re.match(r'^\d+_\d+$',r['name'] or '') and
 ↪  4<=int(r['name'].split('_')[0])<=7]
agree=tot=0; mirror_out=0; flat_rim=0; rows=[]
for nm in names:
    M=snappy.Manifold(nm)
    if M.solution_type()!='all tetrahedra positively oriented': continue
    zg=np.array([complex(s) for s in M.tetrahedra_shapes('rect')]); sols=solve_all(M)
    if not any(np.max(abs(s-zg))<1e-6 for s in sols): sols.append(zg)
    info=[]
    for s in sols:
```

```
        G=(s-w)/(s-np.conj(w)); info.append((np.sum(abs(G)**2),sum(D(x) for x in
 ↪  s),np.max(abs(s-zg))<1e-6,np.max(abs(s-np.conj(zg)))<1e-6,np.all(abs(s.imag)<1e-9),np.max(abs(G))))
    gi=[i for i,x in enumerate(info) if x[2]][0]; near=int(np.argmin([x[0] for x in info]));
 ↪  big=int(np.argmax([x[1] for x in info]))
    tot+=1; agree+=(near==gi)
    mi=[x for x in info if x[3]]; mirror_out+= bool(mi) and all(x[5]>1 for x in mi)
    rows.append((nm,len(sols),near==gi,big==gi,sorted([(round(x[0],3),round(x[1],3)) for x in
 ↪  info])[:4]))
print('knots: %d   true geometry = nearest the vacuum: %d   (= max volume: %d)'%(tot,agree,sum(r[3] for r
 ↪  in rows)))
print('mirror solutions found sitting outside the disk: %d knots'%mirror_out)
for r in rows[:8]: print('  %-5s solutions %2d  nearest=true %s  maxvol=true %s  (sum|G|^2, volume)
 ↪  smallest: %s'%r)

ns=[r[1] for r in rows]; print('solutions found per knot: min %d, median %d, max
 ↪  %d'%(min(ns),int(np.median(ns)),max(ns)))
print('flat (all-real) solutions found:',sum(1 for nm in rows for _ in [0]))
```

winding_cs_01.py

```
# SCRIPT: WINDING-CS-01
# Does the turn book (Chern-Simons) live in winding integers? Neumann's flattening formula (Geom. Topol.
 ↪  2004):
# give each tetrahedron log-parameters w0 = log z + p*pi*i, w1 = log z' + q*pi*i, w2 = -w0-w1 (z' =
 ↪  1/(1-z)),
# with integers p, q chosen so every edge's and both cusp curves' log-parameters sum to 0. Then
#   i(Vol + i CS) = sum_j R(z_j; p_j, q_j)  mod pi^2,  R = Li2(z) + 1/2 log z log(1-z) + (pi i/2)(q log z
 ↪  + p log(1-z)) - pi^2/6.
# Why this tests it: Vol needs no integers (the continuous part gives it, as SHAPE-DISK found); if CS
 ↪  truly sits in the
# windings, the formula with the solved integers must reproduce snappy's CS, and dropping the integer
 ↪  term must lose it.
import numpy as np, snappy, warnings, re, database_knotinfo as d
from cypari import pari
from mpmath import polylog, mpc
warnings.filterwarnings('ignore')
def solve_int(A,b):
    M=pari('['+';'.join(','.join(str(int(x)) for x in r) for r in A)+']');
 ↪  B=pari('['+','.join(str(int(x)) for x in b)+']~')
    s=pari.matsolvemod(M,0,B)
    return None if s==0 else [int(s[i]) for i in range(len(A[0]))]
def R(z,p,q):
    lz=np.log(z); l1=np.log(1-z)
    return complex(polylog(2,mpc(z)))+0.5*lz*l1+0.5j*np.pi*(q*lz+p*l1)-np.pi**2/6
out=[]
names=[r['name'] for r in d.link_list() if re.match(r'^\d+_\d+$',r['name'] or '') and
 ↪  4<=int(r['name'].split('_')[0])<=9]
for nm in names:
    M=snappy.Manifold(nm)
    if M.solution_type()!='all tetrahedra positively oriented': continue
    z=np.array([complex(s) for s in M.tetrahedra_shapes('rect')]); n=len(z); zp=1/(1-z)
    G=M.gluing_equations(); E=np.array(G.list(),dtype=int).reshape(-1,3*n)   # rows: edges..., meridian,
 ↪  longitude
    a,b,c=E[:,0::3],E[:,1::3],E[:,2::3]                    # interleaved (a,b,c) per tetrahedron;
 ↪  checked below
    zpp=1-1/z; chk=np.exp(a@np.log(z)+b@np.log(zp)+c@np.log(zpp))
```

```
    if np.max(abs(chk[:-2]-1))>1e-6: a,b,c=E[:,:n],E[:,n:2*n],E[:,2*n:]
    L=(a-c)@np.log(z)+(b-c)@np.log(zp)
    tgt=np.zeros(len(E),dtype=complex)                    # flattening: every edge and cusp curve sums to
↪   0 (Zickert Def 6.1)
    rhs=(tgt-L)/(1j*np.pi)
    if np.max(abs(rhs-np.round(rhs.real)))>1e-6: out.append((nm,'nonint')); continue
    A=np.hstack([a-c,b-c]); sol=solve_int(A.tolist(),np.round(rhs.real).astype(int).tolist())
    if sol is None: out.append((nm,'nosol')); continue
    p,q=np.array(sol[:n]),np.array(sol[n:])
    tot=sum(R(z[j],p[j],q[j]) for j in range(n)); vol=tot.imag; cs=-tot.real
    cont=-sum((complex(polylog(2,mpc(zj)))+0.5*np.log(zj)*np.log(1-zj)-np.pi**2/6).real for zj in z)
    cv=complex(M.complex_volume())
    def m(x,per): return (x+per/2)%per-per/2
    per=np.pi**2/2                                        # compare modulo pi^2/2 (convention-safe)
   out.append((nm,vol,cv.real,m(cs-cv.imag,per),m(cont-cv.imag,per),int(np.sum(abs(p))+np.sum(abs(q)))))
ok=[o for o in out if len(o)==6]; bad=[o for o in out if len(o)==2]
print('knots solved: %d ; no integer flattening found: %d %s'%(len(ok),len(bad),bad[:5]))
V=np.array([o[1]-o[2] for o in ok]); E1=np.array([o[3] for o in ok]); E0=np.array([o[4] for o in ok])
print('volume from formula vs snappy: max |diff| = %.2e'%np.max(abs(V)))
print('CS WITH winding integers, mismatch mod pi^2/2: max %.2e, median
↪   %.2e'%(np.max(abs(E1)),np.median(abs(E1))))
print('CS WITHOUT winding term (continuous part only): median |mismatch| %.3f, as share of the period:
↪   %.0f%%'%(np.median(abs(E0)),100*np.median(abs(E0))/(np.pi**2/4)))
print('examples (knot, total |p|+|q|):',[(o[0],o[5]) for o in ok[:8]])
off=np.pi**2/6
def allow(x):  # distance to the nearest of 0, +pi^2/6, -pi^2/6 modulo pi^2/2 (weak-flattening ambiguity,
↪   Neumann 2004)
    w=lambda y:abs((y+np.pi**2/4)%(np.pi**2/2)-np.pi**2/4)
    return np.minimum(np.minimum(w(x),w(x-off)),w(x+off))
b1=allow(E1); b0=allow(E0)
print('CS WITH windings (up to the pi^2/6 ambiguity): max mismatch %.2e over %d knots; exact without any
↪   allowance: %d'%(b1.max(),len(b1),int(np.sum(abs(E1)<1e-8))))
print('CS from the continuous part only, same allowance: median mismatch %.3f, max %.3f, matched to 1e-6:
↪   %d of %d'%(np.median(b0),b0.max(),int(np.sum(b0<1e-6)),len(b0)))
```

drawing_poly_01.py

```
# SCRIPT: DRAWING-POLY-01
# Step 1: the knot potential read straight off the DRAWING (Cho-Kim-Kim, arXiv 1301.6219, Sec. 2).
# One variable z_k per side of the diagram. Each crossing with sides (a,b,c,d) adds
#   P = Li2(z_b/z_a) - Li2(z_b/z_c) + Li2(z_d/z_c) - Li2(z_d/z_a).
# Hyperbolicity equations exp(z_k dV/dz_k) = 1 are branch-free products of (1 - ratio)^(+-1).
# Modified potential V0 = V - sum_k (z_k dV/dz_k) log z_k ; at the geometric solution V0 = i(vol + i cs)
↪   mod pi^2.
# Check 1 (this script): the paper's own figure-eight potential (8 sides, 4 crossings) must give vol(4_1)
↪   = 2.02988.
import numpy as np
from mpmath import polylog, mpc
from scipy.optimize import least_squares
def build(crossings):                  # crossings: list of (a,b,c,d) side indices -> list of (eps, p, q)
↪   for eps*Li2(z_p/z_q)
    T=[]
    for a,b,c,dd in crossings: T+= [(+1,b,a),(-1,b,c),(+1,dd,c),(-1,dd,a)]
    return T
def zdV(T,z,n):                         # z_k dV/dz_k with principal logs
    g=np.zeros(n,complex)
```

```
    for e,p,q in T:
        L=np.log(1-z[p]/z[q]); g[p]+=-e*L; g[q]+=e*L
    return g
def V0(T,z,n):
    V=sum(e*complex(polylog(2,mpc(z[p]/z[q]))) for e,p,q in T)
    return V-np.sum(zdV(T,z,n)*np.log(z))
def H(T,z,n):                          # exp(z_k dV/dz_k) - 1, branch-free
    return np.exp(zdV(T,z,n))-1
def solve(T,n,starts=300,seed=1):
    rng=np.random.default_rng(seed); sols=[]
    def f(x):
        z=np.concatenate([[1.0+0j],x[:n-1]+1j*x[n-1:]]); h=H(T,z,n); return
 ↪  np.concatenate([h.real,h.imag])
    for _ in range(starts):
        x0=rng.normal(0,2,2*(n-1))
        try: r=least_squares(f,x0,xtol=1e-15,ftol=1e-15,gtol=1e-15,max_nfev=4000)
        except Exception: continue
        if r.cost<1e-20:
            z=np.concatenate([[1.0+0j],r.x[:n-1]+1j*r.x[n-1:]])
            ratios=[z[p]/z[q] for e,p,q in T]
            if min(min(abs(u),abs(1-u)) for u in ratios)>1e-6: sols.append(z)
    return sols
if __name__=='__main__':
    fig8=[(0,5,1,6),(5,2,4,1),(7,3,6,4),(2,0,3,7)]   # from the paper's V, 0-based: [z6/z1, z6/z2, z7/z2,
 ↪  z7/z1] -> (a,b,c,d)=(1,6,2,7) etc.
    T=build(fig8); sols=solve(T,8)
    vals=sorted({(round(V0(T,z,8).imag,6),round(((-V0(T,z,8).real+np.pi**2/2)%np.pi**2)-np.pi**2/2,5))
 ↪  for z in sols})
    print('figure-eight from the drawing: solutions found %d ; distinct (Im V0 = vol, cs mod
     ↪  pi^2):'%len(sols),vals)
    print('expected vol(4_1) = 2.029883, cs = 0')
```

drawing_poly_02.py

```
# SCRIPT: DRAWING-POLY-02
# Read the knot potential off ANY drawing (KnotInfo PD code). PD crossing X[i,j,k,l]: i incoming under
 ↪  strand, then
# counter-clockwise; opposite pairs (i,k) under, (j,l) over. The Cho-Kim-Kim crossing term pairs (b,d)
 ↪  and (a,c) as
# opposite sides. The paper's corner placement is in a figure we do not have, so every placement is
 ↪  tested: which pair
# is (b,d), and the order within each pair, separately for positive and negative crossings (8 x 8 = 64
 ↪  rules).
# A rule is accepted only if, on several different knots, the solution with the largest Im V0 reproduces
 ↪  SnapPy's
# complex volume (vol, and cs mod pi^2) exactly. No fitting: either a rule reproduces it or it does not.
import numpy as np, snappy, warnings, itertools, database_knotinfo as d
warnings.filterwarnings('ignore')
from drawing_poly_01 import build, solve, V0
def sign_of(X):
    i,j,k,l=X; return +1 if (j==l+1 or l>j+1) else -1
PLACE=[]
for pair in ('under','over'):
    for s1 in (0,1):
        for s2 in (0,1): PLACE.append((pair,s1,s2))
def place(X,rule):
```

```
    i,j,k,l=X; pair,s1,s2=rule
    bd=(i,k) if pair=='under' else (j,l); ac=(j,l) if pair=='under' else (i,k)
    b,dd=bd if s1==0 else bd[::-1]; a,c=ac if s2==0 else ac[::-1]
    return (a,b,c,dd)
KI={r['name']:r for r in d.link_list()}
def crossings_from(name,rp,rn):
    pd=eval(KI[name]['pd_notation']); labels=sorted({e for X in pd for e in X}); idx={e:t for t,e in
 ↪  enumerate(labels)}
    cr=[]
    for X in pd:
        a,b,c,dd=place(X,rp if sign_of(X)>0 else rn); cr.append((idx[a],idx[b],idx[c],idx[dd]))
    return cr,len(labels)
def score(name,rp,rn,starts=40):
    cr,n=crossings_from(name,rp,rn); T=build(cr); sols=solve(T,n,starts=starts)
    if not sols: return None
    vals=[V0(T,z,n) for z in sols]; best=max(vals,key=lambda v:v.imag)
    cv=complex(snappy.Manifold(name).complex_volume())
    dcs=((-best.real-cv.imag)+np.pi**2/2)%np.pi**2-np.pi**2/2
    return abs(best.imag-cv.real),abs(dcs)
if __name__=='__main__':
  hits=[]
  for rp in PLACE:
    for rn in PLACE:
        s=score('4_1',rp,rn)
        if s and s[0]<1e-6 and s[1]<1e-5:
            s2=score('5_2',rp,rn)
            if s2 and s2[0]<1e-6 and s2[1]<1e-5: hits.append((rp,rn))
  print('rules reproducing 4_1 and 5_2 exactly:',len(hits))
  for h in hits[:8]: print('  positive:',h[0],' negative:',h[1])
  import json; json.dump(hits,open('drawing_rules.json','w'))
```

drawing_poly_05.py

```
# SCRIPT: DRAWING-POLY-05
# From the drawing alone: the spectrum of V0 values, one per solution family = one per boundary-parabolic
 ↪  geometry of the
# knot. Each value is decay (Im V0 = volume of that geometry) + direction (Re V0 = -cs mod pi^2). The
 ↪  largest decay is the
# knot's own volume (Cho). Compare with SnapPy's geometric complex volume and with the ROOTS-DISK volume
 ↪  ladder.
import numpy as np, snappy, warnings
warnings.filterwarnings('ignore')
from drawing_poly_01 import build, solve, V0
from drawing_poly_02 import crossings_from
R=('under',0,1); P2=np.pi**2
for nm in ['4_1','5_2','6_1','6_2','6_3']:
    cr,n=crossings_from(nm,R,R); T=build(cr); sols=solve(T,n,starts=220,seed=7)
    spec=[]
    for z in sols:
        v=V0(T,z,n); key=(round(v.imag,5),round(((-v.real+P2/2)%P2)-P2/2,4))
        if key not in spec: spec.append(key)
    spec.sort(reverse=True); cv=complex(snappy.Manifold(nm).complex_volume())
    print('%-4s from the drawing, (decay = vol, direction = cs) per geometry:'%nm, spec, '| SnapPy:
     ↪  (%.5f, %.4f)'%(cv.real,((cv.imag+P2/2)%P2)-P2/2))
```

knot__volume__01.py

```
# SCRIPT: KNOT-VOLUME-01
# Exploratory. The first meaning of VOTKP (volume of the knot potential), run through the two books.
# Each knot's Jones polynomial is a string of light: let t = e^{i theta} run once around the circle, the way colour runs
# violet to red, and follow V(t) as a track in the plane. Since V(1) = 1, every track leaves the same point and comes back to it.
#   climb book = Mahler measure m(V) = mean of log|V| around the circle (how far out the string climbs)
#   turn book  = how many times the track winds around 0 (the vacuum the closed string encloses)
#              = number of Jones roots inside the unit circle (argument principle, after removing the t^k factor)
# Target: hyperbolic volume of the knot complement (KnotInfo), every hyperbolic knot with 3-12 crossings.
# Baselines: crossing number, and log|det| (Dunfield's observation that log det tracks volume).
import numpy as np, database_knotinfo as d, re
from scipy.stats import spearmanr
th=np.linspace(0,2*np.pi,4097)[:-1]; t=np.exp(1j*th)
rows=[]
for r in d.link_list():
    if not r['volume'] or not r['jones_polynomial_vector']: continue
    try: vol=float(r['volume'])
    except ValueError: continue
    if vol<=0: continue
    v=eval(r['jones_polynomial_vector']); lo,hi,c=v[0],v[1],np.array(v[2:],float)
    p=np.polyval(c[::-1],t)                          # V(t)/t^lo, same |V|, no t^k winding
    mahler=np.mean(np.log(np.abs(p)))
    roots=np.roots(c[::-1]); inside=int(np.sum(np.abs(roots)<1-1e-9));
    onc=int(np.sum(np.abs(np.abs(roots)-1)<1e-9))
    cr=int(re.match(r'(\d+)',r['name']).group(1))
    rows.append((r['name'],cr,vol,mahler,inside,len(c)-1,np.log(int(r['determinant'])),onc))
R=np.array([x[1:] for x in rows],float); cr,vol,mah,ins,span,ldet,onc=R.T
print('hyperbolic knots, 3-12 crossings:',len(rows))
for nm,x in [('crossing number',cr),('Jones span',span),('log det  [Dunfield baseline]',ldet),('climb: Mahler measure of V',mah),('turn: roots inside (vacuum enclosed)',ins),('turn share: inside / span',ins/span)]:
    print('  %-40s rho=%.3f'%(nm,spearmanr(x,vol)[0]))
print('\nwithin each crossing number (removes the size effect): mean rho over 8..12')
for nm,x in [('log det',ldet),('Mahler (climb)',mah),('inside share (turn)',ins/span)]:
    print('  %-22s'%nm,' '.join('%d:%.2f'%(c,spearmanr(x[cr==c],vol[cr==c])[0]) for c in range(8,13)))
# volume left over after log det: does either book carry it?
k=np.polyfit(ldet,vol,1); res=vol-np.polyval(k,ldet)
print('\nafter log det is taken out: climb rho=%.3f  turn share rho=%.3f'%(spearmanr(mah,res)[0],spearmanr(ins/span,res)[0]))
print('Jones roots sitting on the circle: %d knots of %d'%(np.sum(onc>0),len(rows)))
```

knot__volume__03.py

```
# SCRIPT: KNOT-VOLUME-03
# Second opposite for the turn book: count the turn not by roots, but by the signed area the Jones string sweeps
# (the geometric part, as on the Smith disk), scaled by the string's own size (mean |V|^2). Test against volume and |signature|,
# then against the volume left after log det and the Jones climb.
import numpy as np, database_knotinfo as d
from scipy.stats import spearmanr
th=np.linspace(0,2*np.pi,4097)[:-1]; t=np.exp(1j*th)
```

```
X=[]
for r in d.link_list():
    if not r['jones_polynomial_vector']: continue
    try: vol=float(r['volume']); sig=int(r['signature'])
    except (ValueError,TypeError): continue
    if vol<=0: continue
    v=eval(r['jones_polynomial_vector']); c=np.array(v[2:],float); p=np.polyval(c[::-1],t)*t**v[0]
    dz=np.roll(p,-1)-p; area=0.5*np.sum(p.real*dz.imag-p.imag*dz.real)
    X.append((vol,abs(sig),area/np.mean(np.abs(p)**2),np.mean(np.log(np.abs(p))),np.log(int(r[
 ↪  'determinant']))))
vol,asig,ta,mj,ldet=np.array(X).T
print('turn as swept area / size:  vs volume rho=%.3f   vs |signature|
 ↪  rho=%.3f'%(spearmanr(ta,vol)[0],spearmanr(np.abs(ta),asig)[0]))
A=np.column_stack([ldet,mj,np.ones_like(vol)]); c,*_=np.linalg.lstsq(A,vol,rcond=None); res=vol-A@c
print('vs volume left after log det + climb: rho=%.3f'%spearmanr(ta,res)[0])
```

homfly_books_02.py

```
# SCRIPT: HOMFLY-BOOKS-02
# Opposite of HOMFLY-BOOKS-01. The knot-theory surface inside HOMFLY is not |v|=|z|=1: the sl(N) lines
 ↪  are
# v = q^-N... in KnotInfo's convention the Jones line is v = t^-1, z = t^1/2 - t^-1/2 (checked in -01).
 ↪  So the sl(N) line is
# v = t^-(N/2)... use v = t^(-s/2), z = t^1/2 - t^-1/2 with s = N (s=2 Jones; s=0 Alexander, Conway z), t
 ↪  = e^{i theta}.
# (1) climb on each line s = 0..8 vs volume and beyond log det + Jones climb; (2) the whole surface (s
 ↪  continuous 0..8,
# and the 'light surface' v = e^{ia}, z = 2i sin(b/2)) as a 2-D climb.
import numpy as np, sympy as sp, database_knotinfo as d, warnings
from scipy.stats import spearmanr
warnings.filterwarnings('ignore')
v,z=sp.symbols('v z')
th=np.linspace(1e-3,2*np.pi-1e-3,1025); t=np.exp(1j*th); sl=np.exp(0.5j*th); Zt=sl-1/sl
A=np.linspace(0,2*np.pi,97)[:-1]; Va,Bb=np.meshgrid(np.exp(1j*A),A,indexing='ij'); Zs=2j*np.sin(Bb/2)
S=np.arange(0,9)
rows=[]
for r in d.link_list():
    if not r['homfly_polynomial']: continue
    try: vol=float(r['volume']); det=int(r['determinant'])
    except (ValueError,TypeError): continue
    if vol<=0: continue
    f=sp.lambdify((v,z),sp.sympify(r['homfly_polynomial'].replace('^','**')),'numpy')
    lines=[np.mean(np.log(np.abs(f(np.exp(-0.5j*s*th),Zt))+1e-300)) for s in S]
    surf=np.mean(np.log(np.abs(f(Va,Zs))+1e-300))
    rows.append([vol,np.log(det),surf]+lines)
R=np.array(rows); vol,ld,surf=R[:,0],R[:,1],R[:,2]; L=R[:,3:]
def r2(cols):
    X=np.column_stack(cols+[np.ones_like(vol)]); c,*_=np.linalg.lstsq(X,vol,rcond=None); p=X@c; return
 ↪  1-np.sum((vol-p)**2)/np.sum((vol-vol.mean())**2)
jones=L[:,2]
print('knots',len(R))
print(' sl(N) line  climb vs vol   R^2 with log det   R^2 with log det + Jones climb')
for i,s in enumerate(S): print('   N=%d        %+.3f          %.3f
 ↪  %.3f'%(s,spearmanr(L[:,i],vol)[0],r2([ld,L[:,i]]),r2([ld,jones,L[:,i]])))
print('all lines N=0..8 together with log det: R^2 %.3f'%r2([ld]+[L[:,i] for i in range(len(S))]))
```

```
print('light surface (v on circle, z = 2i sin(b/2)) climb vs vol rho=%.3f ; R^2 with log det %.3f ; with
 ↪  log det + Jones %.3f'%(spearmanr(surf,vol)[0],r2([ld,surf]),r2([ld,jones,surf])))
```

circuit_poly_01.py

```
# SCRIPT: CIRCUIT-POLY-01  (first attempt at our own knot polynomial)
# The knot diagram as an electrical circuit, read with light.
#  - Checkerboard-shade the diagram's regions. Shaded regions are the nodes; each crossing is a branch
 ↪  joining the two
#    shaded regions that meet there (the Tait graph). Kirchhoff: the number of spanning trees of this
 ↪  network is the
#    knot determinant for alternating diagrams (checked below on every alternating knot).
#  - Each branch's element is set by the crossing's handedness: right-handed = admittance z, left-handed
 ↪  = 1.
#    So Q(z) = sum over spanning trees T of z^(number of right-handed branches in T). Q(1) = tree count.
#  - Light leads: z = e^{i phi} runs once around the circle. Climb = Mahler measure of Q. Turn = swept
 ↪  area / size.
# Checks: (A) tree count = det on alternating knots; (B) invariance: same knot, different diagrams
 ↪  (snappy backtrack);
# (C) what the books carry: volume (beyond log det) and signature.
import numpy as np, database_knotinfo as d, snappy, re, warnings, random
from fractions import Fraction
from scipy.stats import spearmanr
warnings.filterwarnings('ignore')
def sign_of(X):
    i,j,k,l=X
    return +1 if (j==l+1 or l>j+1) else -1     # KnotTheory PositiveQ convention (wrap handled by l>j+1)
def tait(pd):
    darts={}                                   # edge label -> list of (crossing, position)
    for c,X in enumerate(pd):
        for p,e in enumerate(X): darts.setdefault(e,[]).append((c,p))
    def twin(c,p):
        a,b=darts[pd[c][p]]; return b if a==(c,p) else a
    seen={}; faces=[]
    for c in range(len(pd)):
        for p in range(4):
            if (c,p) in seen: continue
            f=len(faces); faces.append([]); h=(c,p)
            while h not in seen:
                seen[h]=f; c2,p2=twin(*h); faces[f].append((c2,p2)); h=(c2,(p2+1)%4)
    corner={}                                  # (crossing, corner index p = between p and p+1) -> face
    for f,cs in enumerate(faces):
        for (c,p) in cs: corner[(c,p)]=f
    n=len(pd); assert len(faces)==n+2, 'Euler check failed'
    adj={f:set() for f in range(len(faces))}
    for c in range(n):
        for p in range(4): adj[corner[(c,p)]].add(corner[(c,(p+1)%4)])
    col={0:0}; st=[0]
    while st:
        f=st.pop()
        for g in adj[f]:
            if g not in col: col[g]=1-col[f]; st.append(g)
    edges=[]
    for c in range(n):
        a,b=(corner[(c,0)],corner[(c,2)]) if col[corner[(c,0)]]==0 else (corner[(c,1)],corner[(c,3)])
        if a!=b: edges.append((a,b,sign_of(pd[c])))
```

```
    nodes=sorted({f for f in col if col[f]==0}); idx={f:i for i,f in enumerate(nodes)}
    return len(nodes),[(idx[a],idx[b],s) for a,b,s in edges]
def det_int(M):                                 # Bareiss, exact integers
    M=[r[:] for r in M]; n=len(M); prev=1; sg=1
    for k in range(n-1):
        if M[k][k]==0:
            for r in range(k+1,n):
                if M[r][k]!=0: M[k],M[r]=M[r],M[k]; sg=-sg; break
            else: return 0
        for i in range(k+1,n):
            for j in range(k+1,n): M[i][j]=(M[i][j]*M[k][k]-M[i][k]*M[k][j])//prev
        prev=M[k][k]
    return sg*M[n-1][n-1] if n else 1
def tree_poly(nv,edges):                        # coefficients T_k, k = right-handed branches in the tree
    m=nv-1
    if m<=0: return [1]
    vals=[]
    for x in range(m+1):
        L=[[0]*nv for _ in range(nv)]
        for a,b,s in edges:
            w=x if s>0 else 1
            L[a][a]+=w; L[b][b]+=w; L[a][b]-=w; L[b][a]-=w
        vals.append(det_int([r[1:] for r in L[1:]]))
    xs=list(range(m+1)); coef=[Fraction(0)]*(m+1)   # Lagrange -> monomial coefficients
    for i,xi in enumerate(xs):
        num=[Fraction(1)]; den=Fraction(1)
        for j,xj in enumerate(xs):
            if j==i: continue
            num=[Fraction(0)]+num;
            for t in range(len(num)-1): num[t]-=xj*num[t+1]
            den*=(xi-xj)
        for t in range(len(num)): coef[t]+=vals[i]*num[t]/den
    c=[int(v) for v in coef]
    while len(c)>1 and c[-1]==0: c.pop()
    while len(c)>1 and c[0]==0: c.pop(0)
    return c
TH=np.exp(1j*np.linspace(0,2*np.pi,2049)[:-1])
def books(c):
    p=np.polyval(np.array(c[::-1],float),TH); dz=np.roll(p,-1)-p
    return float(np.mean(np.log(np.abs(p)+1e-300))),
     ↪  float(0.5*np.sum(p.real*dz.imag-p.imag*dz.real)/np.mean(np.abs(p)**2))
def Q_of(pd): return tree_poly(*tait([list(X) for X in pd]))
def snapname(nm):
    m=re.match(r'^(\d+)([an])_(\d+)$',nm)
    return 'K%s%s%s'%m.groups() if m else nm
# (A) Kirchhoff check, (C) census
rows=[]; bad=0; alt_ok=0; alt_n=0
for r in d.link_list():
    try: vol=float(r['volume']); sig=int(r['signature']); det=int(r['determinant'])
    except (ValueError,TypeError): continue
    if vol<=0 or not r['pd_notation']: continue
    if int(re.match(r'(\d+)',r['name']).group(1))>12: continue
    pd=eval(r['pd_notation'])
    try: c=Q_of(pd)
    except AssertionError: bad+=1; continue
```

```
    if 'a' in r['name'] or int(re.match(r'(\d+)',r['name']).group(1))<=10 and
     ↪ r.get('alternating','')=='Y':
        pass
    if r.get('alternating')=='Y':
        alt_n+=1; alt_ok+= (sum(c)==det)
    m,t=books(c); rows.append((r['name'],vol,sig,np.log(det),m,t,len(pd),c))
print('(A) Kirchhoff: alternating knots with tree count = det: %d of %d   (diagram failures:
 ↪ %d)'%(alt_ok,alt_n,bad))
vol=np.array([x[1] for x in rows]); sig=np.array([x[2] for x in rows]); ld=np.array([x[3] for x in
 ↪ rows]); cm=np.array([x[4] for x in rows]); ct=np.array([x[5] for x in rows])
print('(C) n=%d  climb vs volume rho=%.3f | log det vs volume
 ↪ rho=%.3f'%(len(rows),spearmanr(cm,vol)[0],spearmanr(ld,vol)[0]))
k=np.polyfit(ld,vol,1); res=vol-np.polyval(k,ld)
print('    climb vs volume left after log det: rho=%.3f'%spearmanr(cm,res)[0])
A=np.column_stack([ld,cm,np.ones_like(vol)]); cc,*_=np.linalg.lstsq(A,vol,rcond=None); p=A@cc
print('    log det + circuit climb: R^2=%.3f (log det alone
 ↪ 0.708)'%(1-np.sum((vol-p)**2)/np.sum((vol-vol.mean())**2)))
print('    turn vs signature rho=%.3f ; |turn| vs |signature|
 ↪ rho=%.3f'%(spearmanr(ct,sig)[0],spearmanr(abs(ct),abs(sig))[0]))
print('    examples:',[(x[0],x[7]) for x in rows[:4]])
# (B) invariance on redrawn diagrams
random.seed(3); test=random.sample(rows,25); same=0; tot=0; cl_same=0
for nm,*_,c0 in test:
    try: L=snappy.Link(snapname(nm))
    except Exception: continue
    ref=Q_of(L.PD_code()); m0=books(ref)[0]
    for s in (4,8):
        L2=L.copy(); L2.backtrack(s); q=Q_of(L2.PD_code()); tot+=1
        same+= (q==ref); cl_same+= abs(books(q)[0]-m0)<1e-6
print('(B) invariance on %d redrawn diagrams: whole polynomial same %d, climb same
 ↪ %d'%(tot,same,cl_same))
```

surfaceology_01.py

```
# SCRIPT: SURFACEOLOGY-01
# Reproduction (not new work) of the core results of Arkani-Hamed, Cao, Dong, Figueiredo, He (2023-24):
#  (1) Tr(phi^3) tree amplitude = sum over triangulations of 1/(product of planar X's): denominators only.
#  (2) Hidden zero: set the non-planar c_{1j} = 0 (j = 3..n-1) and the whole amplitude vanishes, though
 ↪ no single
#      Feynman term does.
#  (3) The delta shift: X(even,even) -> X + delta, X(odd,odd) -> X - delta. The leading term at large
 ↪ delta is the
#      pion (NLSM) amplitude: numerators appear out of shifted denominators. Checked against the known 4-
 ↪ and 6-point
#      NLSM amplitudes, up to one overall constant.
#  (4) The u-variables from words of 2x2 matrices along curves (the Smith/Mobius link): u_C + prod
 ↪ u_D^{cross} = 1.
import itertools, random, sympy as sp
from fractions import Fraction as F
def tris(poly):                                  # triangulations of a polygon (vertex list) as lists of
 ↪ diagonals
    if len(poly)<4: return [[]]
    out=[]; a,b=poly[0],poly[-1]
    for k in range(1,len(poly)-1):
        c=poly[k]
        for L in tris(poly[:k+1]):
```

```
            for R in tris(poly[k:]):
                d=[]
                if k>1: d.append((a,c))
                if k<len(poly)-2: d.append((c,b))
                out.append(L+R+d)
    return out
def amp(n,X):
    s=0
    for T in tris(list(range(1,n+1))):
        p=1
        for (i,j) in T: p*=X[(min(i,j),max(i,j))]
        s+=1/p
    return s
def planar(n): return [(i,j) for i in range(1,n+1) for j in range(i+2,n+1) if not (i==1 and j==n)]
def Xget(X,i,j,n):
    i=(i-1)%n+1; j=(j-1)%n+1
    if i==j or abs(i-j)==1 or {i,j}=={1,n}: return 0
    return X[(min(i,j),max(i,j))]
def c(X,i,j,n): return Xget(X,i,j,n)+Xget(X,i+1,j+1,n)-Xget(X,i,j+1,n)-Xget(X,i+1,j,n)
random.seed(2)
# (2) hidden zero, n = 6: c_{13}=c_{14}=c_{15}=0 fixes X_{2j} in terms of the rest; amplitude should
 ↪  vanish
n=6; syms={k:sp.Symbol('X%d%d'%k) for k in planar(n)}
eqs=[c(syms,1,j,n) for j in (3,4,5)]
sol=sp.solve(eqs,[syms[(2,4)],syms[(2,5)],syms[(2,6)]],dict=True)[0]
A=sp.simplify(sp.together(amp(n,syms)).subs(sol))
print('(2) hidden zero n=6: amplitude on c_13=c_14=c_15=0 ->',A, ' | Feynman
 ↪  terms:',len(tris(list(range(1,7)))))
# (3) delta shift
d=sp.Symbol('delta')
def shifted(n,Xs):
    S={}
    for (i,j),v in Xs.items():
        S[(i,j)]= v+d if (i%2==0 and j%2==0) else (v-d if (i%2==1 and j%2==1) else v)
    return S
X4={k:sp.Rational(random.randint(1,50),7) for k in planar(4)}
lead4=sp.limit(amp(4,shifted(4,X4))*d**2,d,sp.oo); nl4=X4[(1,3)]+X4[(2,4)]
print('(3) n=4: leading shifted term / (X13+X24) =',sp.nsimplify(lead4/nl4))
def nlsm6(X):
    g=lambda i,j:X[(min(i,j),max(i,j))]
    return ((g(1,3)+g(2,4))*(g(4,6)+g(1,5))/g(1,4)+(g(2,4)+g(3,5))*(g(1,5)+g(2,6))/g(2,5)
            +(g(3,5)+g(4,6))*(g(2,6)+g(1,3))/g(3,6)-(g(1,3)+g(2,4)+g(3,5)+g(4,6)+g(1,5)+g(2,6)))
rat=[]
for trial in range(3):
    X6={k:sp.Rational(random.randint(1,60),random.randint(1,9)) for k in planar(6)}
    lead6=sp.limit(sp.together(amp(6,shifted(6,X6)))*d**4,d,sp.oo);
 ↪  rat.append(sp.nsimplify(lead6/nlsm6(X6)))
print('    n=6: leading shifted term / NLSM_6, three random points =',rat)
# (4) u-variables as cross-ratios of ordered points z_1<...<z_n on a line (the worldsheet form;
 ↪  Mobius-invariant,
#     the quantity a Smith-chart move preserves). u_ij =
 ↪  (z_i-z_{j-1})(z_{i-1}-z_j)/((z_i-z_j)(z_{i-1}-z_{j-1})).
#     Check the binary relations u_C + prod_{D crossing C} u_D = 1, and Mobius invariance, for n = 5..8.
import numpy as np
def us(z,n):
```

```
    Z=lambda k: z[(k-1)%n]
    return {(i,j):(Z(i)-Z(j-1))*(Z(i-1)-Z(j))/((Z(i)-Z(j))*(Z(i-1)-Z(j-1))) for (i,j) in planar(n)}
def crosses(a,b):
    (i,j),(k,l)=a,b
    return (i<k<j<l) or (k<i<l<j)
worst=0; mob=0
for n in range(5,9):
    z=np.sort(np.random.default_rng(n).uniform(0,10,n)); U=us(z,n)
    for C in U: worst=max(worst,abs(U[C]+np.prod([U[D] for D in U if crosses(C,D)])-1))
    a,b,cc,dd=1.3,0.4,-0.2,2.0                  # a Mobius map (det>0, keeps order on the chosen range)
    z2=(a*z+b)/(cc*z+dd); o=np.argsort(z2); U2=us(z2[o],n) if np.all(np.diff(z2)>0) else U
    mob=max(mob,max(abs(U[k]-U2[k]) for k in U))
print('(4) u-equations, n=5..8: worst |u_C + prod u_D - 1| =',worst,' | Mobius change in u:',mob)
```

film__phase__01.py

```
# SCRIPT: FILM-PHASE-01
# Exploratory. P1: bulk metals the eye matches (CIE 1931 2 deg, D65, CIEDE2000) but whose Smith tracks
 ↪  differ in phase
# (arg Gamma). Put the same transparent film (n = 1.5, thickness 10-300 nm) on each and compute the
 ↪  colours.
# Reading: does the film turn the hidden phase difference back into colour? Compare against the
 ↪  phase-track gap.
import sys, numpy as np, colour
sys.path.insert(0,'/home/claude/votkp_research'); import ri_load
S=ri_load.element_sets(); nonmetal={'B','C','Ge','Se','Si','Te'}
lv=np.arange(380,781,5)/1000.0
cmfs=colour.MSDS_CMFS['CIE 1931 2 Degree Standard Observer']; ill=colour.SDS_ILLUMINANTS['D65']
shape=colour.SpectralShape(380,780,5)
def lab_of(R):
    sd=colour.SpectralDistribution(dict(zip((lv*1000).astype(int),R)))
    XYZ=colour.sd_to_XYZ(sd,cmfs,ill,shape=shape)/100
    return colour.XYZ_to_Lab(XYZ,colour.CCS_ILLUMINANTS['CIE 1931 2 Degree Standard Observer']['D65'])
data={}
for el,d in S.items():
    if el in nonmetal: continue
    o=np.argsort(d['lam']); N=np.interp(lv,d['lam'][o],d['n'][o])+1j*np.interp(lv,d['lam'][o],d['k'][o])
    data[el]=N
def film_R(N,dnm,nf=1.5):
    r01=(1-nf)/(1+nf); r12=(nf-N)/(nf+N); delta=2*np.pi*nf*dnm/(lv*1000)
    r=(r01+r12*np.exp(2j*delta))/(1+r01*r12*np.exp(2j*delta)); return np.abs(r)**2
bulk={el:lab_of(np.abs((1-N)/(1+N))**2) for el,N in data.items()}
phase={el:np.angle((1-N)/(1+N)) for el,N in data.items()}
els=sorted(data); pairs=[]
for i,a in enumerate(els):
    for b in els[i+1:]:
        dE=colour.delta_E(bulk[a],bulk[b],method='CIE 2000')
        if dE<2.0:
            dph=np.degrees(np.mean(np.abs(np.angle(np.exp(1j*(phase[a]-phase[b]))))))
            fe=[colour.delta_E(lab_of(film_R(data[a],t)),lab_of(film_R(data[b],t)),method='CIE 2000')
 ↪  for t in np.arange(10,301,10)]
            pairs.append((a,b,dE,dph,max(fe),np.median(fe)))
print('bulk-matched metal pairs (bulk dE2000 < 2):',len(pairs))
print(' pair     bulk dE   mean phase gap (deg)   film dE max   film dE median')
for p in sorted(pairs,key=lambda x:x[3]): print(f' {p[0]:>2}-{p[1]:<2}   {p[2]:6.2f}      {p[3]:8.2f}
 ↪  {p[4]:8.2f}       {p[5]:8.2f}')
```

```
if len(pairs)>3:
    from scipy.stats import spearmanr
    r=spearmanr([p[3] for p in pairs],[p[4] for p in pairs]); print('\nrank link phase gap -> film colour gap (max over thickness): rho =',round(r.correlation,3),' p =',round(r.pvalue,4))
```

torus_check_02.py

```
# SCRIPT: TORUS-CHECK-02
# Is 15n_41185 the torus knot T(4,5)? Without Sage, compare fingerprints with T(4,5) built from its braid (s1 s2 s3)^5:
# (1) hyperbolicity: torus-knot complements are not hyperbolic (no positive-volume solution);
# (2) homology of the first several cyclic covers (a strong knot fingerprint), H1 of the 2..7-fold cyclic branched-type covers
#     via M.covers(n, cover_type='cyclic').
import snappy, warnings
warnings.filterwarnings('ignore')
A=snappy.Manifold('K15n41185')
B=snappy.Link(braid_closure=[1,2,3]*5).exterior()
C=snappy.Manifold('10_124'); D=snappy.Link(braid_closure=[1,2]*5).exterior()
for nm,M in [('15n_41185',A),('T(4,5) braid',B),('10_124',C),('T(3,5) braid',D)]:
    hc=[]
    for n in range(2,7):
        try: hc.append(str([X.homology() for X in M.covers(n,cover_type='cyclic')]))
        except Exception as e: hc.append('err')
    print('%-13s solution: %-40s vol %.4f'%(nm,M.solution_type(),float(M.volume())))
    print('    cyclic cover homology n=2..6:',hc)
```

paper_figs_01.py

```
# SCRIPT: PAPER-FIGS-01
# Figures for the VOTKP paper: (1) the vacuum disk with knot tetrahedra, element tracks, the racket rotation and the
# |G|^2 = 1/2 line; (2) the vacuum bowl, per-tetrahedron volume against |G|^2 with the exact series; (3) the spectrum read
# off drawings (decay vs direction) incl. torus knots; (4) HOMFLY sl(N) light lines.
import numpy as np, snappy, warnings, re, sys, database_knotinfo as d, matplotlib
matplotlib.use('Agg'); import matplotlib.pyplot as plt
from mpmath import polylog
warnings.filterwarnings('ignore'); sys.path.insert(0,'/home/claude/votkp_research'); import ri_load
plt.rcParams.update({'font.family':'serif','font.size':9,'axes.linewidth':0.7})
w=np.exp(1j*np.pi/3); V0=1.0149416064096536; k2=3*np.sqrt(3)/4
def D(z): return float(np.imag(complex(polylog(2,z))))+np.angle(1-z)*np.log(abs(z))
G=lambda z:(z-w)/(z-np.conj(w))
pts=[]
for r in d.link_list():
    nm=r['name'] or ''
    if not re.match(r'^\d+_\d+$',nm) or not 4<=int(nm.split('_')[0])<=8: continue
    M=snappy.Manifold(nm)
    if M.solution_type()!='all tetrahedra positively oriented': continue
    for s in M.tetrahedra_shapes('rect'): s=complex(s); pts.append((G(s),D(s)))
g=np.array([p[0] for p in pts]); dv=np.array([p[1] for p in pts])
S=ri_load.element_sets(); lv=np.linspace(0.38,0.78,81); nonm={'B','C','Ge','Se','Si','Te'}
fig,ax=plt.subplots(figsize=(4.6,4.6))
th=np.linspace(0,2*np.pi,400)
ax.plot(np.cos(th),np.sin(th),'k',lw=0.9); ax.plot(np.sqrt(.5)*np.cos(th),np.sqrt(.5)*np.sin(th),'k--',lw=0.5)
```

```
for el,dd in S.items():
    o=np.argsort(dd['lam']);
 ↪  N=np.interp(lv,dd['lam'][o],dd['n'][o])+1j*np.interp(lv,dd['lam'][o],dd['k'][o]); Gm=(1-N)/(1+N)
    ax.plot(Gm.real,Gm.imag,color='#b5651d' if el not in nonm else '#2a7f62',lw=0.6,alpha=0.8)
ax.scatter(g.real,g.imag,s=3,color='#1f3b73',alpha=0.5,lw=0,label='knot tetrahedra (4-8 crossings)')
z0=0.3+1.1j; tri=[G(z0),G(1/(1-z0)),G(1-1/z0)]
for a in tri: ax.plot([0,a.real],[0,a.imag],color='#8b1a1a',lw=0.8)
ax.scatter([t.real for t in tri],[t.imag for t in tri],s=18,color='#8b1a1a',zorder=5,label='one
 ↪  tetrahedron, three axis labels')
ax.scatter([0],[0],marker='*',s=80,color='k',zorder=6)
ax.text(0.03,-0.09,'vacuum',fontsize=8); ax.text(-0.98,0.9,'ground / rim',fontsize=8)
ax.plot([],[],color='#b5651d',label='metal reflection tracks');
 ↪  ax.plot([],[],color='#2a7f62',label='non-metal tracks')
ax.plot([],[],'k--',lw=0.5,label='|G|$^2$ = 1/2'); ax.set_aspect('equal'); ax.set_xlim(-1.05,1.05);
 ↪  ax.set_ylim(-1.05,1.05); ax.axis('off')
ax.legend(loc='lower left',fontsize=6.5,frameon=False); fig.tight_layout();
 ↪  fig.savefig('/home/claude/paper/fig1_disk.pdf');
 ↪  fig.savefig('/home/claude/paper/fig1_disk.png',dpi=130)
fig,ax=plt.subplots(figsize=(4.6,3.2))
x=np.abs(g)**2; ax.scatter(x,dv,s=3,color='#1f3b73',alpha=0.4,lw=0,label='tetrahedra of 4-8 crossing
 ↪  knots')
xs=np.linspace(0,0.7,200); ax.plot(xs,V0-k2*xs,'k',lw=1,label='bowl: $V_0-\\frac{3\\sqrt{3}}{4}|G|^2$')
for ph,ls in [(0,'-'),(np.pi/3,'--')]:
    rr=np.sqrt(xs); gg=rr*np.exp(1j*ph);
 ↪  ax.plot(xs,V0-k2*xs+np.sqrt(3)/4*(gg**3).real+3*np.sqrt(3)/16*xs**2,color='#8b1a1a',lw=0.9,ls=ls)
ax.plot([],[],color='#8b1a1a',label='with cubic + quartic terms (two directions)')
ax.set_xlabel('$|G|^2$  (squared distance from the vacuum point)'); ax.set_ylabel('volume of the piece
 ↪  $D(z)$'); ax.set_xlim(0,0.7); ax.set_ylim(0.2,1.06)
ax.legend(fontsize=6.5,frameon=False); fig.tight_layout();
 ↪  fig.savefig('/home/claude/paper/fig2_bowl.pdf');
 ↪  fig.savefig('/home/claude/paper/fig2_bowl.png',dpi=130)
spec={'4_1':[(2.02988,0),(-2.02988,0)],'5_2':[(2.82812,-3.0241),(0,1.1135),(-2.82812,-3.0241)],
 '6_1':[(3.16396,3.0789),(1.4151,0.211),(-1.4151,0.211),(-3.16396,3.0789)],
 '6_2':[(4.40083,-3.997),(1.53058,0.3291),(0,2.4011),(-1.53058,0.3291),(-4.40083,-3.997)],
 '6_3':[(5.69302,0),(0.9243,1.8906),(0.9243,-1.8906),(-0.9243,1.8906),(-0.9243,-1.8906),(-5.69302,0)]}
P2=np.pi**2; tor={'3_1':[5/6],'5_1':[1/10,9/10],'7_1':[3/14,5/14,13/14],'8_19':[11/12],'10_124':[14/15]}
fig,ax=plt.subplots(figsize=(4.6,3.3)); cols=['#1f3b73','#8b1a1a','#2a7f62','#b5651d','#5b3f8c']
for (nm,L),c in zip(spec.items(),cols):
    a=np.array(L); ax.scatter(((a[:,1]+P2/2)%P2-P2/2)/P2,a[:,0],s=22,color=c,label=nm,zorder=3)
for i,(nm,fr) in enumerate(tor.items()):
    f=np.array(fr); f=(f+0.5)%1-0.5;
 ↪  ax.scatter(f,np.zeros_like(f),marker='D',s=20,facecolor='white',edgecolor='k',zorder=4,label='torus
 ↪  knots (decay 0)' if i==0 else None)
ax.axhline(0,color='k',lw=0.5); ax.set_xlabel('direction  $\\mathrm{Re}\\,V_0/\\pi^2$  (mod 1)');
 ↪  ax.set_ylabel('decay  $\\mathrm{Im}\\,V_0$ = volume')
ax.legend(fontsize=6.5,frameon=False,ncol=6,loc='upper center',bbox_to_anchor=(0.5,1.16));
 ↪  fig.tight_layout(); fig.savefig('/home/claude/paper/fig3_spectrum.pdf');
 ↪  fig.savefig('/home/claude/paper/fig3_spectrum.png',dpi=130)
N=np.arange(0,9); rho=[0.597,-0.067,0.863,0.819,0.784,0.811,0.842,0.853,0.858];
 ↪  r2=[0.810,0.795,0.795,0.804,0.816,0.820,0.827,0.826,0.830]
fig,ax=plt.subplots(figsize=(4.6,2.8)); ax.plot(N,rho,'o-',color='#1f3b73',lw=1,ms=4,label='climb vs
 ↪  volume (Spearman)')
ax2=ax.twinx(); ax2.plot(N,r2,'s--',color='#8b1a1a',lw=1,ms=3.5,label='$R^2$ for volume: log det + Jones
 ↪  + line N'); ax2.set_ylim(0.78,0.84)
ax.set_xlabel('sl(N) light line inside HOMFLY  (N = 0 Alexander, N = 2 Jones)');
 ↪  ax.set_ylabel('Spearman'); ax2.set_ylabel('$R^2$')
```

```
h1,l1=ax.get_legend_handles_labels(); h2,l2=ax2.get_legend_handles_labels();
 ↪  ax.legend(h1+h2,l1+l2,fontsize=6.5,frameon=False,loc='lower right')
fig.tight_layout(); fig.savefig('/home/claude/paper/fig4_homfly.pdf');
 ↪  fig.savefig('/home/claude/paper/fig4_homfly.png',dpi=130)
print('tetrahedra',len(g))
```

Appendix D. Full script list (in the archive)

- bowl_cubic_01.py
- bowl_law_knots_01.py
- bowl_law_knots_02.py
- bowl_law_tetra_01.py
- branch_cut_01.py
- branch_cut_01b.py
- branch_cut_01c.py
- branch_cut_02.py
- circuit_poly_01.py
- circuit_poly_02.py
- cknot_common.py
- colknot_a1_frame_bowl.py
- colknot_a1b_bowl_opposites.py
- colknot_a2_film_winding.py
- colknot_a2b_winding_threshold.py
- colknot_b1_knot_colours.py
- colknot_b2_knot_colours_controls.py
- drawing_indep_01.py
- drawing_indep_02.py
- drawing_poly_01.py
- drawing_poly_02.py
- drawing_poly_03.py
- drawing_poly_04.py
- drawing_poly_05.py
- drawing_robust_01.py
- drawing_robust_02.py
- drawing_robust_03.py
- drawing_robust_04.py
- drawing_torus_01.py
- drawing_torus_02.py
- drawing_torus_03.py
- film_geo_link_01.py
- film_phase_01.py
- flat_cyclic_check_01.py
- flat_lib_01.py
- flip_key_01.py
- flip_key_02.py
- geo_phase_01.py
- handedness_01.py
- homfly_books_01.py
- homfly_books_02.py
- knot_cs_01.py
- knot_cs_02.py
- knot_volume_01.py
- knot_volume_02.py
- knot_volume_03.py
- mirror_check_01.py
- optics_01.py

- paper_figs_01.py
- pressure_line_01.py
- ri_load.py
- robust_dist_test_01.py
- robust_selftest_01.py
- roots_disk_01.py
- roots_disk_02.py
- roots_ladder_01.py
- roots_ladder_02.py
- shape_disk_01.py
- shape_disk_02.py
- silent_walls_01.py
- strong_flat_01.py
- strong_flat_02.py
- strong_flat_03.py
- strong_flat_04.py
- strong_flat_05.py
- strong_flat_06.py
- strong_flat_07.py
- surfaceology_01.py
- tilt_01.py
- torus_check_01.py
- torus_check_02.py
- winding_cs_01.py